

MATHmania-style Web App: Next.js Build Plan (2-Week Sprint) + Cost Estimate

1. Why Next.js full-stack is the right call here

Going web-only lets you collapse what was three separate systems (Flutter app, backend API, admin panel) into **one Next.js codebase**:

- Student pages and admin pages live in the same app (route-protected, e.g. `/dashboard` vs `/admin`).
- API routes / Server Actions replace a separate backend — no second service to build, deploy, or keep in sync.
- One deploy target (Vercel), one database, one auth system.

This is the single biggest reason a 2-week timeline becomes plausible at all.

2. Honest scope for two weeks

Two weeks fully supports a **working core product**. It does not realistically support the AI review-queue workflow, bulk CSV school onboarding, or anti-cheat tooling at production quality — those need real testing time. Recommended split:

In the 2-week launch (Weeks 1-2):

- Signup/login, profile
- Practice quiz flow with 60s timer, green/red reveal, "Show reason"
- One contest round with attempt limits
- Leaderboard filtered by grade
- Admin panel: manual question CRUD, basic contest setup, view registered users
- Question bank seeded via AI generation (done by *you* before launch, not built as a live admin feature yet — see below)

Fast-follow (Weeks 3-4, right after launch):

- Live in-admin AI document upload → question generation → review queue
- Bulk CSV student import
- Session-integrity / anti-cheat flags
- Multiple concurrent contest rounds, export tools

This gets you live on schedule with a real, gradeable quiz product, and the AI content pipeline becomes an admin convenience feature shortly after rather than a launch blocker.

3. Architecture (see diagram above)

One Next.js 15 (App Router) application:

- **Frontend:** React Server Components + Client Components for interactive bits (timer, quiz UI)

- **Backend logic:** Server Actions + a handful of Route Handlers (for things like the AI generation trigger, which needs to run as a background job, not block a request)
- **Database:** PostgreSQL (Neon — serverless, scales to zero, pairs natively with Vercel)
- **ORM:** Prisma or Drizzle (Drizzle is lighter and faster to iterate with under time pressure)
- **Auth:** Auth.js (NextAuth) credentials provider, or Clerk if you want to save build days (see cost tradeoff below)
- **File storage:** Vercel Blob (simplest, same platform) for uploaded documents/photos
- **AI:** Anthropic API (Claude) for question generation, called from a Server Action / background job
- **Email:** Resend, for verification and password reset emails

4. Feature-to-page mapping

Feature	Route(s)	Notes
Signup / login	<code>/signup</code> , <code>/login</code>	Fields: name, email, phone, guardian name, guardian phone, school, password, grade
Student dashboard	<code>/dashboard</code>	Rank, best score, attempts chart (Recharts), recent attempts
Practice quiz	<code>/practice/[sessionId]</code>	60s server-enforced timer per question, reveal + "show reason"
Contest quiz	<code>/contest/[roundId]/[sessionId]</code>	Same engine, gated by attempt limits and round windows
Attempt history	<code>/practice/history</code> , <code>/contest/history</code>	Tables matching your screenshots
Leaderboard	<code>/leaderboard</code>	Filter by round + grade, paginated
Profile	<code>/profile</code>	View/edit, matches your screenshot fields
Admin: question bank	<code>/admin/questions</code>	CRUD, filter by grade/topic/status
Admin: document upload	<code>/admin/generate</code>	Upload → background job → questions land in review queue (fast-follow)
Admin: contests	<code>/admin/contests</code>	Create/edit rounds, attempt limits, grade eligibility
Admin: users/schools	<code>/admin/users</code>	Search, view, (fast-follow: bulk import)

5. The 60-second timer, done correctly on web

Client shows a countdown for UX, but the server is the source of truth:

1. When a question is served, record `served_at` server-side.
2. On submit, server checks `now - served_at <= 60s (+small buffer)`. Late submissions are recorded as timeouts, not accepted answers.
3. This also means a student can't just pause the browser tab to "freeze" the timer — the server doesn't care about client-side time.

6. Data model

Same core schema as the mobile plan (auth, questions, sessions, attempts, contests) — a web frontend doesn't change what you need to store. Key tables: `users`, `schools`, `questions`, `source_documents`, `contests`/`rounds`, `quiz_sessions`, `session_questions`, `attempts`. Keep `session_questions` granular (per-question `served_at/answered_at/selected_option`) — it's what makes the timer enforceable and gives you attempt-replay/audit data for free.

7. 14-day sprint plan

Day	Focus
1	Repo setup, Next.js + Prisma/Drizzle + Neon wired up, deploy skeleton to Vercel, auth scaffolding
2	Signup/login flow fully working (all fields, password hashing, session handling)
3	Database schema finalized and migrated; seed script for test questions
4-5	Practice quiz engine: question serving, timer (client + server enforcement), answer submission, scoring
6	Answer reveal UI (green/red) + "Show reason" expandable explanation
7	Attempt history pages + dashboard chart
8	Contest mode: rounds, attempt limits, round-gated access
9	Leaderboard: ranking query, grade/round filters, pagination
10	Profile page (view/edit)
11	Admin panel: question CRUD, contest management screens
12	AI-assisted question generation — run as a one-off script/CLI job against your source documents to seed the bank (not the full in-admin pipeline yet)
13	QA pass: test full student journey and admin journey end-to-end, fix bugs, mobile-responsive check (web app still needs to work well on phones/tablets students use)
14	Deploy to production domain, smoke test, soft launch

This assumes one focused full-time developer (or two working in parallel on frontend/backend halves). If it's genuinely one person doing this alongside other responsibilities, stretch this to 3 weeks rather than cutting features further — the timer/scoring logic in particular is worth not rushing, since it's what makes the product trustworthy for grading.

8. Cost estimate

8.1 One-time / setup costs

Item	Estimate	Notes
Domain name	~\$12/year	.pk or .com
Initial AI-generated question bank	\$10-25 total	See 8.3 — genuinely this cheap in raw API terms
OCR (if source docs are scanned images)	\$0-50 one-time	Only needed if your source material isn't already digital text; Mathpix or Google Cloud Vision, usage-based — confirm current per-page rates before committing since these change

8.2 Monthly running costs (launch scale — hundreds to a couple thousand students)

Service	Plan	Est. monthly cost
Hosting (Vercel)	Pro, 1-2 seats	\$20-40
Database (Neon Postgres)	Free tier at launch, Pro if you outgrow it	\$0-25
File storage (Vercel Blob)	Free tier (1GB) likely enough at launch	\$0-10
Auth	Auth.js (self-hosted, free) or Clerk if you want to save build days	\$0, or ~\$25+
Email (Resend)	Free tier (3,000 emails/mo)	\$0-20
Error monitoring (Sentry)	Free tier	\$0
AI API (ongoing, light use post-launch)	Pay-as-you-go	~\$5-20
Total		roughly \$25-115/month at launch scale

This scales with usage (mainly database and bandwidth), but for a school-pilot volume you're realistically at the low end of that range. The Vercel seat cost (\$20/seat) is the one guaranteed fixed cost regardless of traffic.

8.3 AI question-generation cost (the number you specifically asked about)

Using current Anthropic API pricing (Claude Sonnet 5, checked today: \$2 input / \$10 output per million tokens):

- A batch call generating ~10 MCQs (question + 4 options + explanation each) from a source chunk runs roughly 1,500 input tokens + 1,800 output tokens.

- Cost per batch: $(1,500 \times \$2 + 1,800 \times \$10) / 1,000,000 \approx \0.021 for 10 questions, or about **\$0.002 per question**.
- A solid starter bank of **5,000 questions** across all grades/topics: roughly **\$10-15** in raw API cost.
- Even a large bank of **10,000-20,000 questions**: roughly **\$20-45**.

The AI compute cost here is genuinely a rounding error in your budget — Anthropic's pricing has come down enough that generating question volume isn't the expensive part. The real cost is **human review time** (a teacher/subject-matter reviewer checking each AI-drafted question before it's published), which you should budget as reviewer hours, not API spend. Batch API usage (50% cheaper, non-urgent processing) can cut the above numbers roughly in half if you're generating in bulk overnight rather than needing instant turnaround.

9. Bottom line

- **Two weeks**: realistic for a working core product (auth, timed practice + one contest round, reveal/reason, leaderboard, basic admin). Not realistic for the full AI review pipeline and bulk-import tooling at production polish — plan those as an immediate fast-follow.
- **Monthly running cost**: roughly \$25-115 at school-pilot scale, dominated by hosting, not AI.
- **AI question generation**: effectively negligible cost (tens of dollars for thousands of questions) — spend your budget on reviewer time instead, since that's what actually protects quality.